UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

EXPERIMENT: Lab 03 - Preemptive SJF comparison with Non-Preemptive SJF

Submitted by:

Jannatul Ferdoushi Islam
Student ID: 23101153
Section: C-1
Semester: 4-1

Submitted to:

Atia Rahman Orthi,
Lecturer,
University of Asia Pacific

Lab Report: Preemptive SJF Comparison with Non-Preemptive SJF

Problem Statement

Implement Preemptive SJF and Non-Preemptive SJF CPU scheduling for the given processes. Calculate Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT and Average WT, then compare the two approaches to determine which one performs better for the given data.

Objective

The objective of this lab is to understand how Preemptive SJF differs from Non-Preemptive SJF. The experiment also shows how process interruption changes CT, TAT and WT, and compares the average turnaround and waiting times of both methods.

Given Data

Process	AT	BT
P1	0	4
P2	1	2
P3	3	2
P4	2	1

Working Principle

Preemptive SJF (Shortest Remaining Time First): Among all arrived processes, the process with the smallest remaining burst time is selected. If a new process arrives with a shorter remaining time, the running process can be interrupted and the new process gets the CPU.

Non-Preemptive SJF: Among all arrived processes, the process with the smallest burst time is selected. Once it starts, it continues until completion; it is not interrupted by a newly arrived shorter process.

For both algorithms: TAT = CT - AT and WT = TAT - BT.

Key Difference

The main difference is interruption. Preemptive SJF may stop a running process when a shorter job arrives, while Non-Preemptive SJF lets the current process finish first. Because of this, the completion order and waiting time can change.

Why Preemptive SJF Can Be Better

Preemptive SJF reacts immediately to newly arrived short jobs. This can reduce the average waiting time and average turnaround time, especially when short jobs arrive while a longer process is running.

Issues of Preemptive SJF

It requires more context switching, so scheduling overhead can increase. Long processes may also face starvation if shorter jobs keep arriving. In addition, the remaining burst time must be known or estimated.

Source Code: Preemptive SJF

```
process = ["p1", "p2", "p3", "p4"]
at = [0,1,3,2]
bt = [4,2,2,1]

n = len(process)

ct = [0] * n
wt = [0] * n
tat = [0] * n

tq= 1
remaining = bt.copy()
time = 0
completed = 0

while completed < n:

    x = -1
    for i in range(n):

        if at[i] <= time and remaining[i] > 0:

            if x == -1 or remaining[i] < remaining[x]:
                x = i

    if x == -1:
        time += 1

    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq

        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1


for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]


print("Process\tAT\tBT\tCT\tTAT\tWT")
```

```
for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i], "\t",
wt[i])

p_avg_tat = sum(tat)/n
p_avg_wt = sum(wt)/n

print("\nAverage TAT =", p_avg_tat)
print("Average WT =", p_avg_wt)
print("\n")



if p_avg_tat < p_avg_tat :
    print ("Preemptive TAT is best.")
else:
    print("Preemptive TAT is best")

if p_avg_wt < p_avg_wt :
    print ("Preemptive WT is best.")
else:
    print("Preemptive WT is best")
```

Source Code: Non-Preemptive SJF

```
process = ["p1", "p2", "p3", "p4"]
at = [0, 1, 3, 2]
bt = [4, 2, 2, 1]

n = len(process)
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [0] * n

time = 0
completed = 0

while completed < n:
    x = -1

    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time += 1
    else:
        time += bt[x]
        ct[x] = time
        done[x] = 1
        completed += 1

for i in range(n):
```

```
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process  AT  BT  CT  TAT  WT")
for i in range(n):
    print(process[i], at[i], bt[i], ct[i], tat[i], wt[i])

np_avg_tat = sum(tat) / n
np_avg_wt = sum(wt) / n

print("Average TAT =", np_avg_tat)
print("Average WT =", np avg wt)
```

Output: Preemptive SJF

```
Process AT  BT  CT  TAT WT
p1   0   4   9   9   5
p2   1   2   3   2   0
p3   3   2   6   3   1
p4   2   1   4   2   1

Average TAT = 4.0
Average WT = 1.75


Preemptive TAT is best
Preemptive WT is best
```

Preemptive Average TAT = 4.0

Preemptive Average WT = 1.75

Output :Non-Preemptive SJF

Process	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	2	7	6	4
P3	3	2	9	6	4
P4	2	1	5	3	2

Non-Preemptive Average TAT = 4.75

Non-Preemptive Average WT = 2.5

Comparison between Preemptive and Non-Preemptive SJF

Point	Preemptive SJF	Non-Preemptive SJF
CPU interruption	Allowed	Not allowed
Selection rule	Shortest remaining time	Shortest burst time
Average TAT	4.0	4.75
Average WT	1.75	2.5
Context switching	More	Less
Overhead	Higher	Lower
Response to short new jobs	Immediate	After current process finishes
Starvation	Possible	Possible
Better for this data	Yes	No

Discussion

Based on the process set used, Preemptive SJF achieved an Average TAT of 4.0 and an Average WT of 1.75, while Non-Preemptive SJF resulted in an Average TAT of 4.75 and an Average WT of 2.5. This shows that the preemptive approach outperformed the non-preemptive one on both metrics in this test case.
This improvement occurs because the preemptive scheduler is able to switch instantly to whichever process has the least remaining burst time. That said, this benefit comes at the cost of increased context-switching and scheduling overhead. Non-Preemptive SJF, by contrast, is more straightforward since a running process is never interrupted once it begins execution.

Conclusion

This experiment compared the performance of Preemptive and Non-Preemptive SJF scheduling algorithms by computing CT, TAT, WT, and their respective averages. Based on the results obtained, Preemptive SJF proved more effective, yielding lower Average TAT and Average WT values. The study also highlighted the inherent trade-off involved: preemption leads to quicker handling of short processes but introduces added overhead from switching between tasks.

Github Link:

https://github.com/Riya1153/Operating-System-Lab./tree/main/Lab%203